

My Personal Data Science Document

Markdown Language Assignment 01

Academic Detail	Information
Student Name	Muhammad Ibtsam ul Haq
Roll Number	MSDSF26M008
Program	MS Data Science
Semester	First Semester, Fall 2026
University	University of the Punjab, Lahore
City	Lahore, Pakistan
Submission Date	26 September 2026

About Me

My name is **Muhammad Ibtsam ul Haq**, and I am a first-semester student of **MS Data Science** at the **University of the Punjab, Lahore**. I live in Lahore, Pakistan. I completed my Bachelor of Mathematics at the University of Education during 2018-2022. My mathematical background has developed my interest in probability, statistics, linear algebra, and logical problem-solving. I have worked in mathematics education and IT operations, where I gained experience in explaining difficult ideas, analysing assessment information, and improving technical systems. As an IT Officer at Bravian International School & College, I worked on network monitoring, automation, and time synchronisation. These experiences helped me recognise the value of reliable information when making decisions.

My interests include Data Science, Machine Learning, Artificial Intelligence, Cloud Computing, and DevOps. I use technologies such as Python, Cloud Computing, SQL, HTML, CSS, and JavaScript, and I want to develop stronger skills in data analysis and machine learning. I am learning Data Science to connect mathematical reasoning with practical computing and solve problems in education and technology. My future career goal is to become a data science and AI professional who builds useful and intelligent systems. I also want to keep improving my communication skills so that I can explain technical findings clearly to different audiences.

My Technology Skills

The levels below are conservative self-assessment descriptions based on my practical work. I distinguish tools I have used from libraries I still plan to learn.

Technology	My Level	Experience	I Use It For
Python	Basic with practical exposure	Monitoring and scripting projects	Automation and backend tasks
SQL	Basic with practical exposure	Network monitoring project	Storing and querying records
HTML, CSS & JavaScript	Basic with practical exposure	Full-stack monitoring system	Application logic and interfaces
C++	Foundational	Programming coursework and practice	Logic and algorithm practice
Linux	Working knowledge	Administration and automation labs	Users, permissions, and services
Bash	Working knowledge	Linux automation script	Repetitive administration tasks
AWS	Associate-certified; hands-on	Cloud architecture and deployment labs	Hosting and infrastructure
Docker	Basic with practical exposure	Containerised application project	Reproducible application environments
Git/GitHub	Working knowledge	CI/CD and project workflows	Version control and collaboration
Microsoft Excel	Working knowledge	Reporting and assessment analysis	Tables, calculations, and reports

My Learning Journey

1. Began my Bachelor of Mathematics at the University of Education in 2018.
2. Developed foundations in probability, statistics, and linear algebra during my degree.
3. Completed my mathematics degree in 2022.
4. Applied analytical thinking in mathematics teaching and assessment analysis.
5. Expanded my technical learning through Linux, Bash, and cloud-computing practice.
6. Earned the AWS Certified Solutions Architect - Associate certification.
7. Practised application deployment, Docker containers, and GitHub Actions workflows.
8. Joined the Amal Academy fellowship in 2025 and developed communication and teamwork skills.
9. Applied programming to network monitoring, IT automation, and load-testing at Bravian in 2026.
10. Entered the MS Data Science program at the University of the Punjab in 2026.

My Learning Areas

- Mathematics and analysis
 - Probability, statistics, and linear algebra
 - Assessment-data interpretation
- Programming and automation
 - Python, HTML, CSS, JavaScript, C++, and SQL
 - Bash scripting and Linux administration
- Cloud and deployment
 - AWS infrastructure
 - Docker and GitHub Actions
 - Terraform and Ansible
- Data science development goals
 - Pandas and NumPy
 - Data visualisation and machine learning

My Favorite Technologies

1. Python

Why I like it: Python lets me express ideas with readable code and connect programming with mathematical problem-solving.

Where I have used it: I used Python in a network monitoring system and in scripting projects.

Useful resource: [Official Python Documentation](#)

2. AWS

Why I like it: AWS helps me understand how applications, networks, storage, and security work together in a cloud environment.

Where I have used it: My projects include a three-tier web application, a multi-VPC security architecture, and static website hosting with S3 and CloudFront.

Useful resource: [AWS Getting Started Documentation](#)

3. Docker

Why I like it: Docker makes application environments easier to reproduce and gives me practical experience with deployment.

Where I have used it: I containerised a basic Python/Node application and practised a multi-container setup with Docker Compose.

Useful resource: [Docker Get Started](#)

4. Git and GitHub

Why I like them: I find version history useful for understanding changes, recovering earlier work, and organising development.

Where I have used them: I configured a GitHub Actions workflow to run tests and build Docker images.

Useful resource: [Pro Git Book](#), by Scott Chacon and Ben Straub.

5. Linux

Why I like it: Linux gives me direct control over services, permissions, and automation through the command line.

Where I have used it: I wrote Bash automation for user creation, permission setup, and package installation, and used cron for scheduling.

Useful resource: [The Linux Kernel Documentation](#), an advanced reference for understanding Linux internals.

Project Checklist

Project: Load-Balancer Capacity Test for a School Network

Problem: An IT team needs to know how many requests per second a service can handle before response time and failure rate become unacceptable. Without this, a busy period can interrupt school operations.

Project idea: I used httpperf to send increasing request rates against a load-balanced backend and recorded response time, success rate, and error counts at each target rate. This builds directly on my IT monitoring experience.

Approach used: I ran repeated httpperf sessions at rising target rates against a real host, logged the results, checked data quality, and calculated summary statistics before drawing conclusions.

Current Assignment Progress

- ☒ Described a project idea connected to my IT experience.
- ☒ Defined the fields for the dataset.
- ☒ Collected a real dataset from an authorised test environment.
- ☒ Included ten real records for Markdown practice.
- ☒ Calculated mean response time and request success rate.
- ☒ Included a chart of the real records.
- ☐ Clean and analyse the full dataset (all rate levels) using Pandas.
- ☐ Repeat tests to check whether the observations are consistent across days.
- ☐ Build and evaluate a predictive model for the capacity threshold.
- ☐ Validate findings before recommending operational changes.

My Dataset

Data origin: These are **real** records from httpperf load tests against host 192.168.98.12, run on 25 September 2026. The full log contains 41 test runs; the ten rows below are a representative sample spanning the tested rate range, from a target of 400 req/s up to 60,000 req/s.

Target rate (req/s)	Planned requests	Successful replies	Mean response (ms)	Success rate (%)
400	12,000	12,000	4.7	100.00
500	15,000	15,000	32.9	100.00
600	18,000	15,944	938.3	88.58
700	21,000	15,064	1178.7	71.73
800	24,000	14,933	1222.9	62.22
1,000	30,000	14,941	1238.3	49.80
2,000	40,000	10,095	1256.4	25.24
5,000	100,000	11,601	1059.3	11.60
10,000	200,000	11,525	1123.8	5.76
60,000	1,200,000	10,705	1169.6	0.89

Data dictionary: Target rate is the request rate httpperf was asked to sustain; planned requests is the number of requests attempted at that rate; successful replies is the number that completed; mean response is the average reply time for that test; success rate is successful replies as a percentage of planned requests.

Interpretation: The backend holds 100% success up to about 500 req/s. Past roughly 600 req/s, response time jumps sharply and the success rate falls, dropping below 1% once the target reaches 60,000 req/s. This is a real capacity limit for this host, not a modelled or assumed one — though ten runs at one point in time are not enough to guarantee the same threshold on a different day or under a different load pattern.

Mathematical Expressions

1. Mean of the Ten Test-Level Response Times

$$\bar{x} = \frac{\sum_{i=1}^n x_i}{n}$$

Here, each x_i is one test's mean response time and $n = 10$.

Calculation: $(4.7 + 32.9 + 938.3 + 1178.7 + 1222.9 + 1238.3 + 1256.4 + 1059.3 + 1123.8 + 1169.6) / 10 = 922.5$ ms.

This is an unweighted average of ten test means. It is not the mean response time across all individual requests, because the tests contain very different numbers of requests.

2. Request Success Rate

$$\text{Success rate} = \frac{N - F}{N} \times 100\%$$

Here, N is the total number of planned requests and F is the number of failed requests, summed across the ten selected tests.

Calculation: $N = 1,660,000$; successful replies = 131,808; $F = 1,528,192$.

Overall success rate: $(131,808 / 1,660,000) \times 100 = 7.94\%$, rounded to two decimal places.

(This low figure is dominated by the extreme 60,000 req/s test, which was deliberately run past the server's capacity to find the breaking point — it is not the success rate under normal load.)

A Small Python Example

```
response_ms = [4.7, 32.9, 938.3, 1178.7, 1222.9, 1238.3, 1256.4, 1059.3, 1123.8, 1169.6]
mean_response = sum(response_ms) / len(response_ms)

planned = [12000, 15000, 18000, 21000, 24000, 30000, 40000, 100000, 200000, 1200000]
replies = [12000, 15000, 15944, 15064, 14933, 14941, 10095, 11601, 11525, 10705]
success_rate = sum(replies) / sum(planned) * 100

print(f"Mean of test means: {mean_response:.1f} ms")
print(f"Request success rate: {success_rate:.2f}%")
```

Project Image

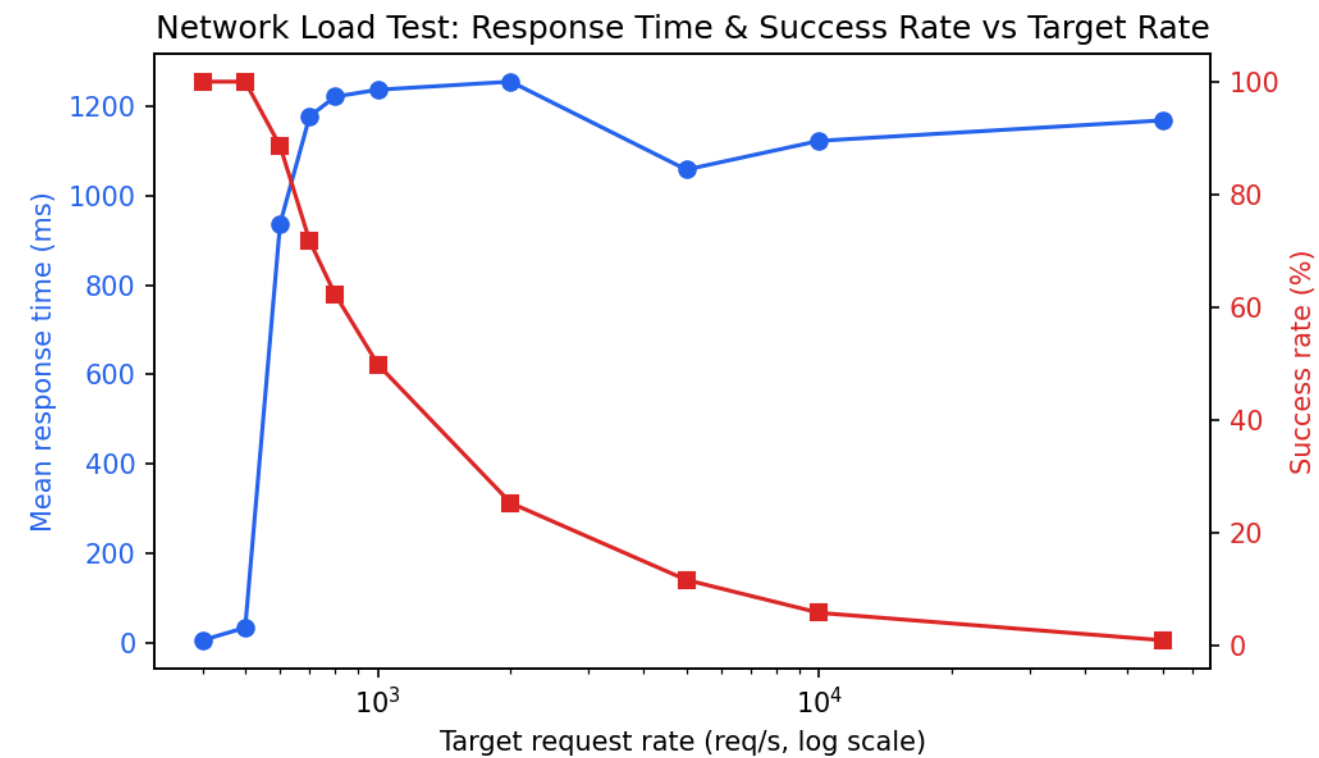


Figure 1. Response time and success rate versus target request rate, from real *httperf* load-test records.

Blockquotes

The following are original learning statements written for this assignment, rather than quotations attributed to public figures.

Understanding a result matters more than producing a number without knowing what it means.

A useful technical solution begins with a clear problem and improves through careful testing.

My Reflection

Preparing this document connects my background in mathematics and IT with the communication skills needed in Data Science. Headings help me organise ideas, while tables make skills and numerical records easier to compare. Inline code distinguishes technical names from ordinary text, and a fenced code block preserves the layout of a program. A checklist also helps me separate completed work from future plans.

Working with the real `httpperf` records instead of an invented dataset was a useful check on my own reasoning: the numbers did not always move smoothly, and I had to decide which of the 41 available test runs to keep so the table stayed readable without hiding the trend. An average of test means is different from an average across all requests, and a real chart still cannot prove that the same limit holds on another day. My next step is to analyse the full 41-row log with Pandas and repeat the test to see whether ~500-600 req/s is a stable threshold. For the viva, I need to be able to modify the Markdown myself and explain the syntax I used.

My learning approach is moving from ~~only collecting results~~ to **checking, understanding, and explaining results**.

Markdown Features I Practiced

- ☒ Level-one heading with #.
- ☒ Level-two headings with ##.
- ☒ Level-three headings with ###.
- ☒ Bold text with **text**.
- ☒ Italic text with *text*.
- ☒ Inline code with backticks.
- ☒ Fenced code block with a Python language label.
- ☒ Ordered list for learning milestones.
- ☒ Unordered list for learning areas.
- ☒ Nested list for related subtopics.
- ☒ Hyperlinks to online resources.
- ☒ Image syntax with meaningful alternative text.
- ☒ Tables for academic details, skills, and data.
- ☒ Blockquotes using >.
- ☒ Completed and incomplete task-list items.
- ☒ Mathematical expressions using LaTeX math blocks.
- ☒ Horizontal rule using ---.
- ☒ Strikethrough using ~~text~~.